Project 2: Grading Rubric

No Credit

- Non submitted assignments
- Assignments late by more than 48 hours (*without exceptions*)
- Non compiling assignments
- Non-independent work
- "Hard coded" solutions
- Unclear or unintelligible code

How will my assignment be graded?

- Grading will be divided into two portions:
  - Manual/Automatic Testing (90%): To assess the correctness of programs.
  - Manual Inspection (10%): A checklist of features your programs should exhibit. These comprise things that cannot be easily checked via unit tests such as good variable name selection, proper decomposition of a problem into multiple functions or cooperating objects, overall design elegance, and proper asymptotic complexity. These features will be checked by graders and assigned credit based on level of compliance. See the remainder of this document for more information.
- You CANNOT get points (even style/manual-inspection points) for code that doesn't compile or for submitting just the files given to you. You CAN get manual inspection points for code that (a) compiles and (b) is an "honest attempt" at the assignment but does not pass any unit tests.
- "Off the top" points refer to items that will lose you points rather than earn you points.
- Extra credit for early submissions:
  - 1% extra credit rewarded for every 24 hours your submission made before the due time;
  - Up to 5% extra credit will be rewarded;
  - Your latest submission before the due time will be used for grading and extra credit checking. You CANNOT choose which one counts.

Manual/Automated Testing Rubric

For this assignment, a portion of the automated testing will be based on JUnit tests and a manual run of your program. The JUnit tests used for grading will NOT be provided for you (you need to test your own programs!), but the tests will be based on what has been specified in the project description and the comments in the code templates. A breakdown of the point allocations is given on the next page:

Automatic Testing Breakdown:

8 pts	Part 1 (Squares)
20 pts	Part 2 (Square List – no sorting)
7 pts	Part 3 (Windows)
25 pts	Part 4 (Window Stack – no sorting)
30 pts	Part 5 (Sorting)

Automatic Testing "Off the Top" Points

Note: "Off the top" points are items that will *lose* you points rather than earn you points.

Always -5 pts	Not passing code style check
Always -5 pts	Not passing JavaDoc style check
Always -5 pts	Compiler Warnings OR using @SuppressWarnings in non-permitted places
Always -100%	Use of "non-permitted code".
Always -100%	Using arrays in a no-arrays project.

Manual Code Inspection Rubric

Inspection Point	Points	High (all points)	Med (1/2 points)	Low (no points)
Code Documentation	-5pts "Off the top"	The entire code base is well documented with meaningful comments in JavaDoc format. Occasional in-method comments used for clarity. [AND] Code has good, meaningful variable, method, and class names.	The code base has some comments, but is lacking comments on some classes/ methods/fields or the comments given are mostly "translating" the code. [AND/OR] Names are mostly meaningful, but a few are unclear or ambiguous (to a human reader)	The only documentation is what was in the template and/or documentation is missing from the code (e.g. taken out). [AND/OR] Names often have single letter identifiers and/or incorrect/meaningless identifiers. (Note: i/j/k acceptable for indexes.)
Big-O Requirements	10pts	The Big-O requirements for listed methods are met and/or exceeded. Code is very efficient.	The Big-O requirements are sometimes met and sometimes not. Code could be more efficient.	The Big-O requirements are never (or almost never) met. Code is extremely inefficient.